FIAP ON

RAÍZES DA INTELIGÊNCIA - PREPARANDO O TERRENO

ENTENDENDO A CIÊNCIA DE DADOS

7

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Página de download do R site oficial 6
Figura 2 - Escolha do servidor para o download do R 7
Figura 3 - Escolha do servidor para download do R 7
Figura 4 - Interface do RStudio 9
Figura 5 - Inserção da sintaxe e execução do código 10
Figura 6 - Exemplo de código RStudio 11
Figura 7 - Utilizando o R como uma calculadora no console do RStudio 12

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Exemplos básicos de sintaxe 11
Código-fonte 2 - Utilizando o R como uma calculadora 12
Código-fonte 3 - Numeric 12
Código-fonte 4 - Integer 13
Código-fonte 5 - Complex 13
Código-fonte 6 - Adição 13
Código-fonte 7 - Subtração 13
Código-fonte 8 - Multiplicação 13
Código-fonte 9 - Divisão 13
Código-fonte 10 - Igualdade 14
Código-fonte 11 - Diferença 14
Código-fonte 12 - Maior que 14
Código-fonte 13 - Menor que 14
Código-fonte 14 - Maior ou igual a 14
Código-fonte 15 - Menor ou igual a 14
Código-fonte 16 - Criar uma variável em R 14
Código-fonte 17 - Exemplo de uma criação de uma variável em R 14
Código-fonte 18 - Visualização das variáveis definidas no ambiente 15
Código-fonte 19 - Remoção de uma variável específica 15
Código-fonte 20 - Criação de vetores 15
Código-fonte 21 - Criação de um vetor vazio 16
Código-fonte 22 - Acessando elementos específicos de um vetor usando índices .. 16
Código-fonte 23 - Operações aplicadas a vetores 16
Código-fonte 24 - Funções aplicadas a vetores 17
Código-fonte 25 - Manipulação de vetores 17
Código-fonte 26 - Criação de matrizes 18
Código-fonte 27 - Exemplo de criação de uma matriz numérica 3x3 em R 18
Código-fonte 28 - Criação de matrizes vazias preenchidas posteriormente 18
Código-fonte 29 - Elementos da Matriz 18
Código-fonte 30 - Operações em Matrizes 19
Código-fonte 31 - Funções aplicadas a matrizes 19
Código-fonte 32 - Manipulação de matrizes 20
Código-fonte 33 - Instalando pacotes 20
Código-fonte 34 - Carregando pacotes 20
Código-fonte 35 - Explorando pacotes 21
Código-fonte 36 - Listando funções 21
Código-fonte 37 - Atualizando pacotes 21
Código-fonte 38 - Removendo pacotes 22
Código-fonte 39 - Criação de um dataframe em R (sintaxe básica) 22
Código-fonte 40 - Criação de um dataframe simples em R 22
Código-fonte 41 - Tibbles 23
Código-fonte 42 - Criação de um tibble simples em R 23
Código-fonte 43 - Acesso aos elementos de dataframes e tibbles 23
Código-fonte 44 - Manipulação de dataframes e tibbles 24

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 ENTENDENDO A CIÊNCIA DE DADOS 5
1.1 Introdução à linguagem de programação R 6
1.2 Instalação e configuração do ambiente R 6
1.3 Interface de desenvolvimento RStudio 8
1.4 Sintaxe básica do R 10
1.5 Utilizando o R como uma calculadora 11
1.6 Tipos numéricos, operações lógicas e manipulações de variáveis 12
1.7 Vetores 15
1.8 Matrizes em R 17
1.9 Instalação e Manipulação de Pacotes 20
1.10 Dataframes e tibbles 22
REFERÊNCIAS 25

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 ENTENDENDO A CIÊNCIA DE DADOS

A linguagem R foi criada por Ross Ihaka e Robert Gentleman na Universidade de Auckland, na Nova Zelândia, no início dos anos 1990. Inicialmente, a linguagem era utilizada principalmente como uma versão aprimorada do software estatístico S (Ihaka & Gentleman, 1996). O objetivo principal de Ihaka e Gentleman era fornecer uma linguagem de programação de código aberto, gratuita e voltada para análise estatística e gráficos. O R foi desenvolvido com base na linguagem S, mantendo sua sintaxe e muitas de suas funcionalidades, porém, com algumas melhorias significativas.

Devido à natureza de código aberto e comunidade ativa de desenvolvedores, o R cresceu rapidamente em popularidade ao longo dos anos. A linguagem ganhou destaque na comunidade acadêmica, entre pesquisadores e cientistas de dados, e tornou-se uma das principais ferramentas utilizadas em análise estatística e ciência de dados. Atualmente, o R é amplamente adotado em diversas áreas, como ciências sociais, biologia, finanças, marketing, entre outras. A linguagem continua evoluindo com a contribuição de uma comunidade global de desenvolvedores, que criam e mantêm pacotes e recursos adicionais para estender a funcionalidade do R.

No contexto da ciência de dados, a linguagem R se destaca pelo seu foco especializado em análise estatística e visualização de dados. Ao compará-lo com outras linguagens de programação amplamente utilizadas, como Python, é possível observar diferenças distintas. Enquanto o Python é uma linguagem de propósito geral, o R foi projetado especificamente para análise de dados e estatística. Essa ênfase no domínio da ciência de dados torna o R uma escolha popular entre estatísticos, pesquisadores e cientistas de dados que trabalham com análise estatística, modelagem e visualização. Outra diferença é a ênfase do R na produção de gráficos e visualizações de dados de alta qualidade. Vale mencionar que a linguagem R possui pacotes especializados, como ggplot2, que permitem criar gráficos estatísticos sofisticados e personalizados com facilidade. Essa capacidade de visualizar dados de forma eficaz é essencial para a exploração e comunicação dos resultados obtidos na análise de dados.

O R possui uma vasta coleção de pacotes e bibliotecas especializadas projetadas para a análise de dados, oferecendo uma ampla variedade de funcionalidades estatísticas avançadas prontas para uso, tendo mais de 19800 pacotes disponíveis. Esses pacotes facilitam análises como regressão, análise multivariada, séries temporais, modelagem

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

preditiva e muito mais. Além disso, o R possui uma comunidade ativa que contribui para o desenvolvimento contínuo de pacotes e recursos específicos para a ciência de dados.

1.1 Introdução à linguagem de programação R

A linguagem de programação R é uma ferramenta utilizada em ciência de dados e análise estatística e oferece funcionalidades e bibliotecas especializadas que permitem aos usuários manipular, visualizar e analisar dados de forma eficiente. O R é uma linguagem de código aberto e possui uma comunidade ativa de desenvolvedores, o que resulta em uma coleção de pacotes que expandem ainda mais suas capacidades.

1.2 Instalação e configuração do ambiente R

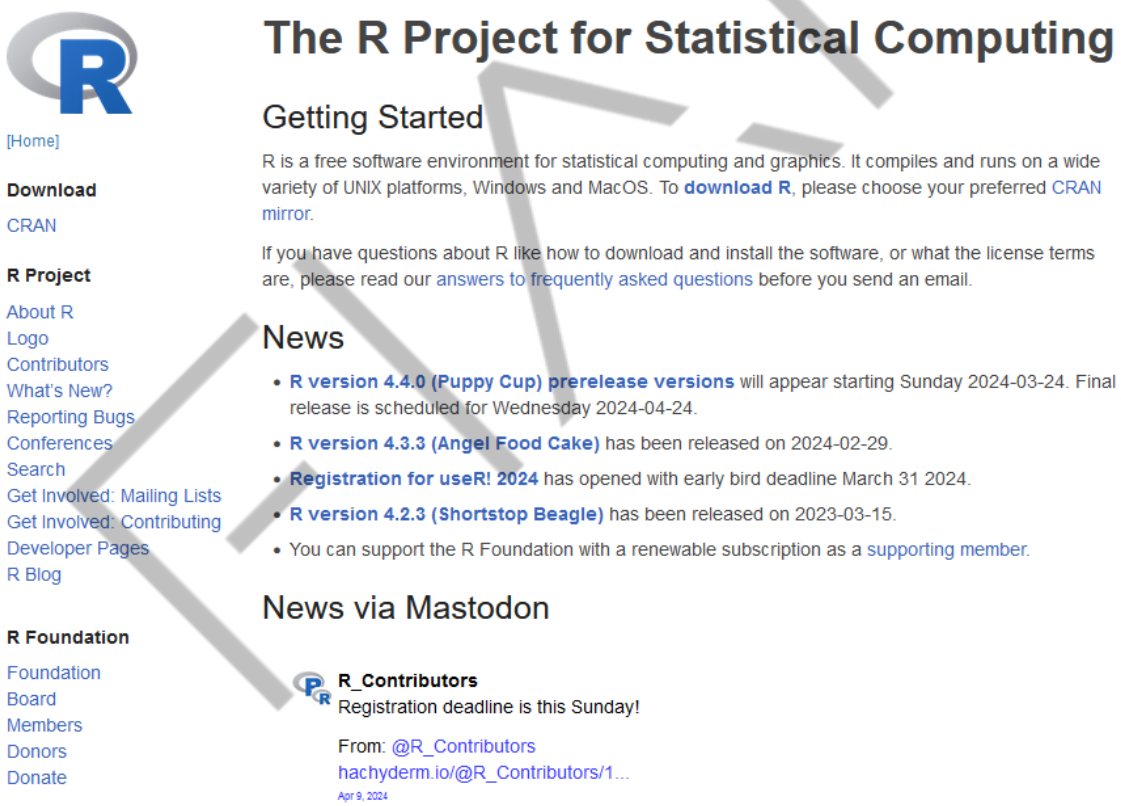

Figura 1 - Página de download do R site oficial
Fonte: site R Project.org (2023)

Para começar a trabalhar com o R, é necessário instalar o software e configurar o ambiente de desenvolvimento. O R está disponível para diferentes sistemas operacionais e pode ser baixado gratuitamente a partir do site oficial do R <https://www.r-project.org/>. É necessário seguir as instruções de instalação específicas para o seu sistema operacional.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

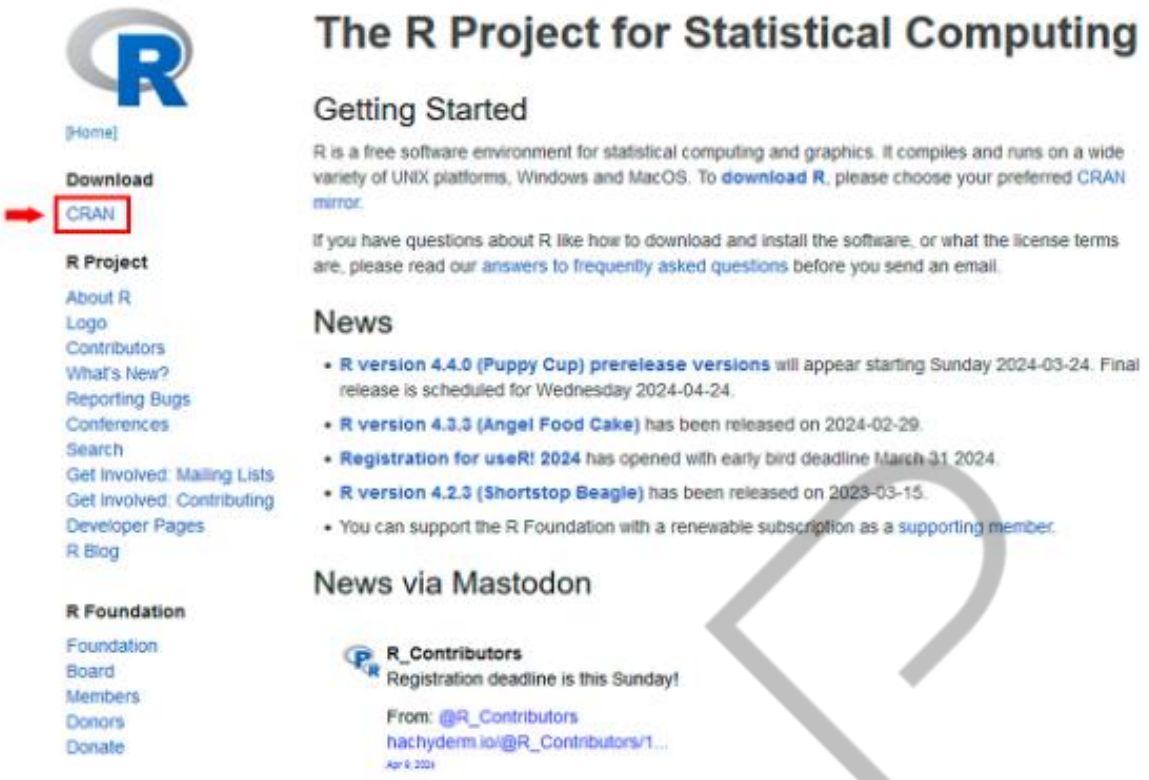

Figura 2 - Escolha do servidor para o download do R
Fonte: site R Project.org (2023)

Conforme recomendado na figura “Página de download do R site oficial”, clique na opção “**CRAN**” para escolher o servidor que usará para baixar o R.

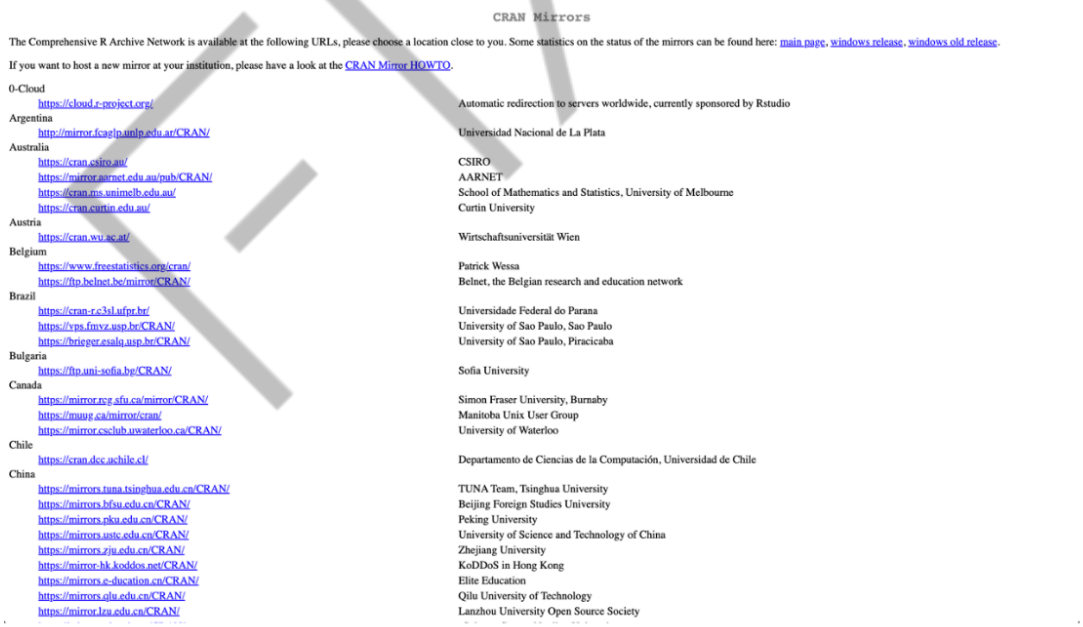

Figura 3 - Escolha do servidor para download do R
Fonte: site R Project.org (2023)

Entre na lista de servidores disponíveis para download do R e escolha aquele que deseja utilizar para iniciar o processo. A recomendação é que a escolha seja de servidores localizados no Brasil. Após a instalação, você pode iniciar o R por meio do seu sistema

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

operacional ou utilizando uma interface de desenvolvimento integrado (IDE), como o RStudio.

1.3 Interface de desenvolvimento RStudio

O R e o RStudio são duas ferramentas essenciais para trabalhar com a linguagem de programação R. Embora o R seja o ambiente de programação principal, o RStudio é uma IDE projetada para facilitar o desenvolvimento e a análise de dados com o R.

O R é uma linguagem de programação estatística e uma das mais populares para análise de dados. É uma plataforma flexível que oferece uma ampla variedade de recursos para manipular, visualizar e modelar dados. O R é um software gratuito e de código aberto, amplamente utilizado pela comunidade acadêmica, cientistas de dados e profissionais de análise de dados em todo o mundo.

O RStudio é uma IDE (Integrated Development Environment) projetada especificamente para trabalhar com a linguagem R. Fornece um ambiente de desenvolvimento amigável e eficiente, reunindo várias ferramentas e recursos que facilitam a escrita, depuração e execução de código R. O RStudio é gratuito e possui versões disponíveis para Windows, macOS e Linux. A principal diferença entre o R e o RStudio é que o R é o ambiente de execução da linguagem R, enquanto o RStudio é uma interface que facilita a interação com o R, oferecendo recursos adicionais para desenvolvimento, depuração e análise de dados.

Principais recursos do RStudio:

1. **Editor de código:** o RStudio oferece um editor de código com recursos avançados, como destaque de sintaxe, autocompletar, formatação automática e indentação o que torna a escrita de código mais eficiente e organizada.
2. **Console do R:** o console do R é uma área no RStudio na qual você pode executar comandos R diretamente e ver os resultados imediatamente. É uma ferramenta útil para testar pequenos trechos de código e realizar tarefas interativas.
3. **Histórico de comandos:** o RStudio mantém um histórico de comandos executados no console do R, permitindo revisitar e reexecutar comandos anteriores sendo útil quando se deseja reproduzir etapas anteriores ou verificar comandos usados anteriormente.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4. **Visualização de variáveis:** o RStudio oferece uma visualização das variáveis existentes no ambiente global do R, permitindo a inspeção e interação com os dados armazenados. É particularmente útil durante o processo de depuração e análise de dados.

5. **Gerenciador de arquivos:** o RStudio possui um gerenciador de arquivos integrado que permite navegar facilmente pelos arquivos e diretórios do projeto e facilita o acesso e a organização de scripts, dados e outros recursos relacionados ao projeto.

6. **Ajuda e documentação:** o RStudio facilita o acesso à documentação e à ajuda relacionada à linguagem R. Ele fornece um painel de ajuda que exibe a documentação do R para funções específicas e pacotes instalados, tornando mais fácil encontrar informações relevantes.

7. ***Plots* e visualizações:** o RStudio possui uma área dedicada para a visualização de gráficos e plots gerados. Permite a exibição e interação com visualizações de dados diretamente na interface do RStudio.

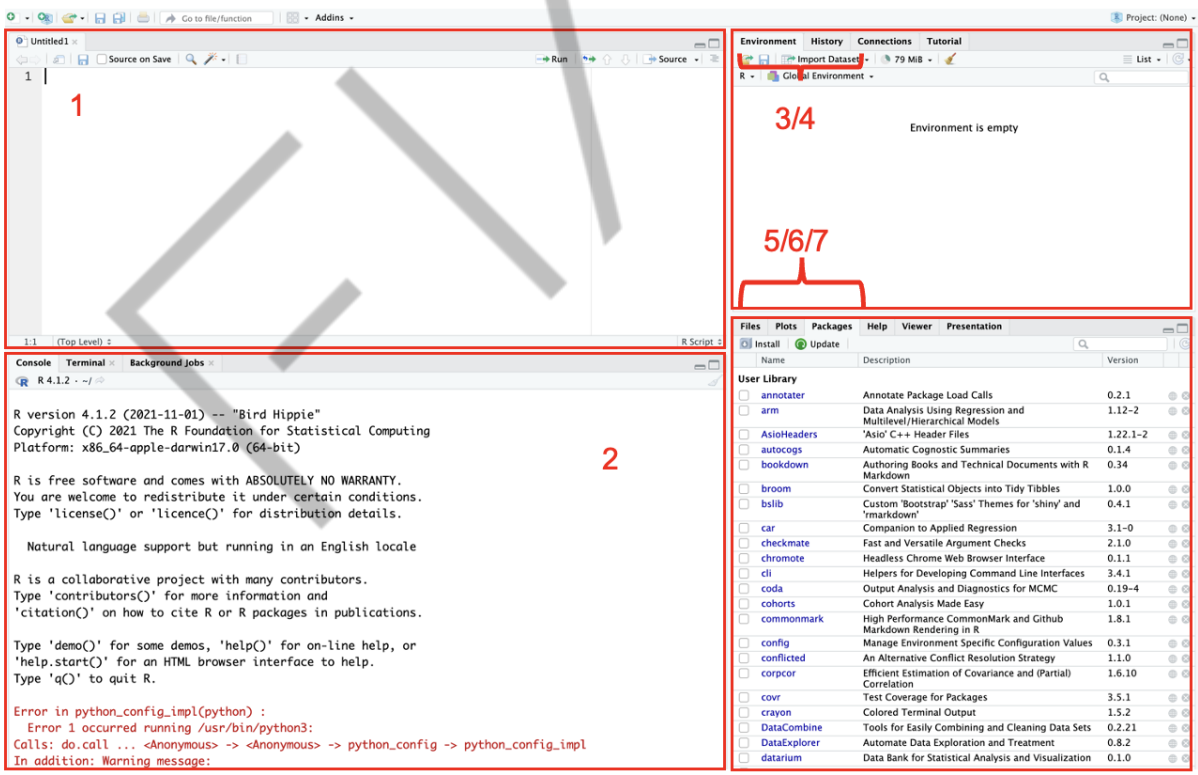

Figura 4 - Interface do RStudio
Fonte: Elaborado pelo autor (2023)

A figura “Interface do RStudio” destaca alguns componentes da interface. O editor de código está no centro, o console do R está na parte inferior esquerda e o painel de

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

visualização de *plots* está na parte inferior direita. Além disso, o gerenciador de arquivos e o painel de ajuda estão disponíveis à direita e fornecem recursos adicionais que facilitam o trabalho com R.

1.4 Sintaxe básica do R

Para iniciar a programação no RStudio, digite a sintaxe proposta no Editor Código, e, em seguida, selecione a opção "**Run**" no lado direito, conforme apresentado na figura "Inserção da sintaxe e execução do código".

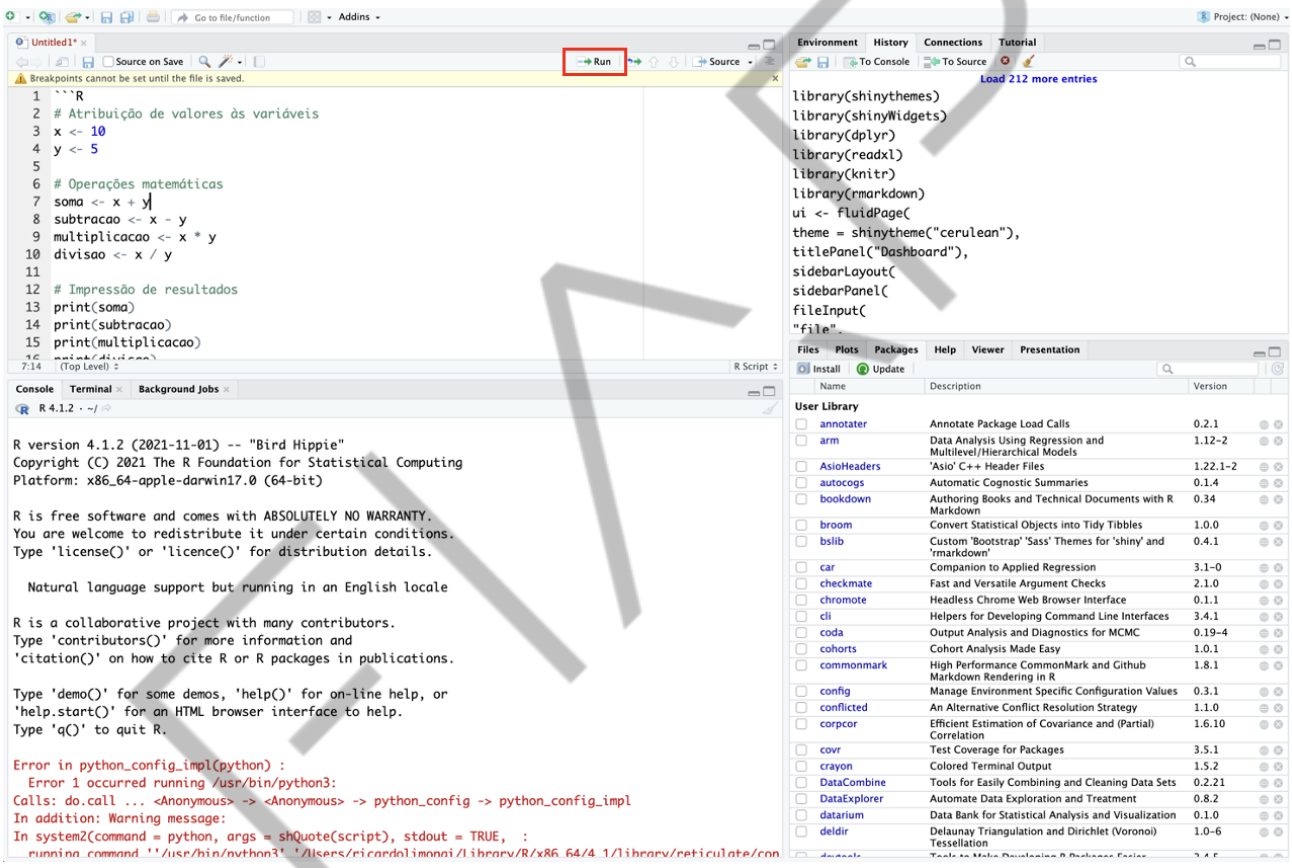

Figura 5 - Inserção da sintaxe e execução do código
Fonte: Elaborado pelo autor (2023)

A sintaxe do R é relativamente simples e fácil de aprender. Os comandos no R são digitados e executados linha por linha. Para melhor aprendizado, é recomendado que copie a sintaxe em itálico no R e execute. Vamos começar com alguns exemplos básicos de sintaxe:

```
# Atribuição de valores às variáveis
x <- 10
y <- 5

# Operações matemáticas
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
soma <- x + y
subtracao <- x - y
multiplicacao <- x * y
divisao <- x / y

# Impressão de resultados
print(soma)
print(subtracao)
print(multiplicacao)
print(divisao)
```

Código-fonte 1 - Exemplos básicos de sintaxe
Fonte: Elaborado pelo autor (2023)

Neste exemplo, atribuímos os valores 10 e 5 às variáveis `x` e `y`, respectivamente. Em seguida, realizamos operações matemáticas simples, como soma, subtração, multiplicação e divisão, utilizando essas variáveis. Por fim, imprimimos os resultados no console.

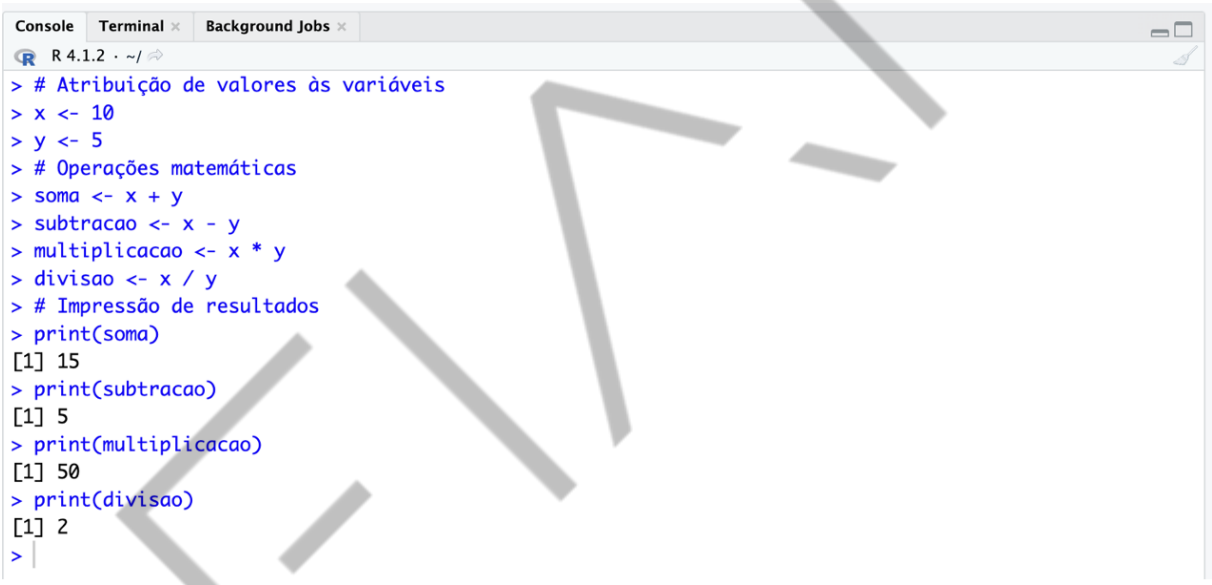

Figura 6 - Exemplo de código RStudio
Fonte: Elaborado pelo autor (2023)

1.5 Utilizando o R como uma calculadora

Uma das maneiras mais simples de começar a utilizar o R é usá-lo como uma calculadora. Podemos realizar operações matemáticas básicas, como adição, subtração, multiplicação e divisão, diretamente no console do R. Observe um exemplo:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Operações matemáticas simples
2 + 3
5 - 2
3 * 4
10 / 2
```

Código-fonte 2 - Utilizando o R como uma calculadora
Fonte: Elaborado pelo autor (2023)

Ao executar esses comandos no console do R, obteremos os resultados das operações diretamente.

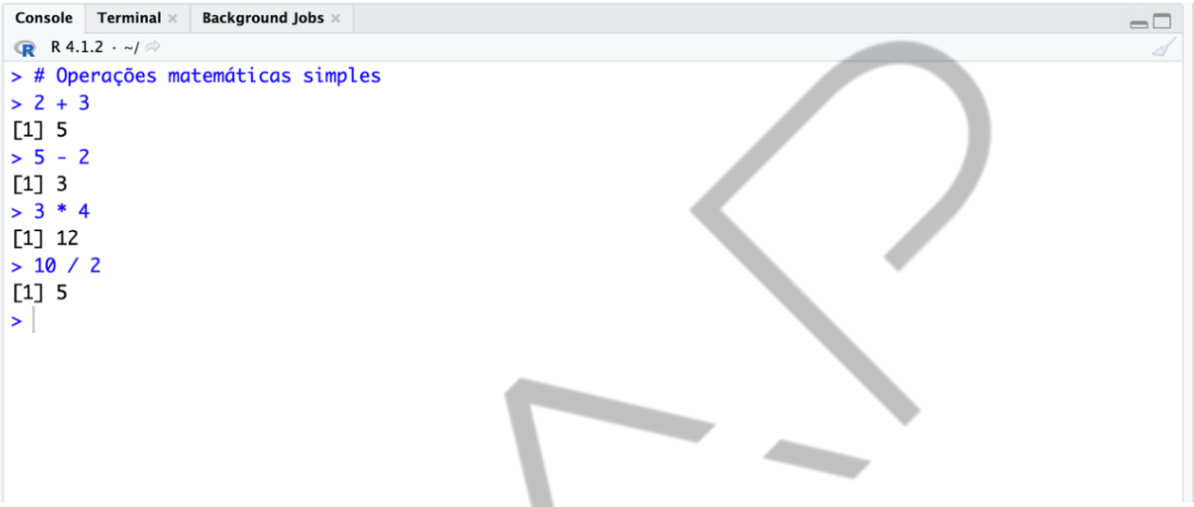

Figura 7 - Utilizando o R como uma calculadora no console do RStudio
Fonte: Elaborado pelo autor (2023)

1.6 Tipos numéricos, operações lógicas e manipulações de variáveis

Ao trabalhar com a linguagem de programação R, é essencial compreender os tipos numéricos disponíveis, as operações lógicas e como criar e manipular variáveis.

Tipos numéricos:

Em R, existem diferentes tipos de dados numéricos que podem ser utilizados para armazenar valores numéricos. Os principais tipos numéricos incluem:

- **Numeric:** é usado para representar números reais (ponto flutuante). Por exemplo:

```
x <- 3.14
y <- 2.718
```

Código-fonte 3 - Numeric
Fonte: Elaborado pelo autor (2023)

- **Integer:** é usado para representar números inteiros. Por exemplo:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
a <- 10
b <- -5
```

Código-fonte 4 - Integer
Fonte: Elaborado pelo autor (2023)

- **Complex:** é usado para representar números complexos, na forma "parte_real + parte_imaginária". Por exemplo:

```
c <- 2 + 3i
d <- -1 - 2i
```

Código-fonte 5 - Complex
Fonte: Elaborado pelo autor (2023)

Operações numéricas:

A linguagem R oferece uma variedade de operações aritméticas que podem ser aplicadas a números. Alguns exemplos de operações numéricas comuns incluem:

- **Adição:** utilize o operador "+".

```
soma <- x + y
```

Código-fonte 6 - Adição
Fonte: Elaborado pelo autor (2023)

- **Subtração:** utilize o operador "-".

```
diferenca <- a - b
```

Código-fonte 7 - Subtração
Fonte: Elaborado pelo autor (2023)

- **Multiplicação:** utilize o operador "*".

```
produto <- a * b
```

Código-fonte 8 - Multiplicação
Fonte: Elaborado pelo autor (2023)

- **Divisão:** utilize o operador "/".

```
quociente <- a / b
```

Código-fonte 9 - Divisão
Fonte: Elaborado pelo autor (2023)

Operações Lógicas:

Em R, também é possível realizar operações lógicas para avaliar condições verdadeiras ou falsas. Os operadores lógicos mais comuns são:

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Igualdade:** utilize o operador "==".

```
x == y
```

Código-fonte 10 - Igualdade
Fonte: Elaborado pelo autor (2023)

- **Diferença:** utilize o operador "!=".

```
a != b
```

Código-fonte 11 - Diferença
Fonte: Elaborado pelo autor (2023)

- **Maior que:** utilize o operador ">".

```
a > b
```

Código-fonte 12 - Maior que
Fonte: Elaborado pelo autor (2023)

- **Menor que:** utilize o operador "<".

```
a < b
```

Código-fonte 13 - Menor que
Fonte: Elaborado pelo autor (2023)

- **Maior ou igual a**:** utilize o operador ">=".

```
a >= b
```

Código-fonte 14 - Maior ou igual a
Fonte: Elaborado pelo autor (2023)

- **Menor ou igual a:** utilize o operador "<=".

```
a <= b
```

Código-fonte 15 - Menor ou igual a
Fonte: Elaborado pelo autor (2023)

Criação de variáveis:

Na linguagem R, é possível criar variáveis para armazenar valores e realizar operações. A sintaxe básica para criar uma variável em R é:

```
nome_variavel <- valor
```

Código-fonte 16 - Criar uma variável em R
Fonte: Elaborado pelo autor (2023)

Por exemplo:

```
idade <- 25
nome <- "João"
```

Código-fonte 17 - Exemplo de uma criação de uma variável em R
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Lista e remoção de variáveis:

Para visualizar as variáveis definidas no ambiente global, podemos usar a função `ls()`.

```
ls()
```

Código-fonte 18 - Visualização das variáveis definidas no ambiente
Fonte: Elaborado pelo autor (2023)

Para remover uma variável específica, utilizamos a função `rm()`.

```
rm(idade)
```

Código-fonte 19 - Remoção de uma variável específica
Fonte: Elaborado pelo autor (2023)

Além disso, podemos remover todas as variáveis com a função `rm(list = ls())` limpando o ambiente global de todas as variáveis definidas.

Esses são os conceitos básicos relacionados aos tipos numéricos, operações lógicas, criação e manipulação de variáveis em R. Com essas ferramentas, você pode realizar cálculos, avaliar condições e armazenar dados de forma eficiente. À medida que você avança na aprendizagem de R, poderá explorar mais recursos e funcionalidades.

1.7 Vetores

Em R, um vetor é uma estrutura de dados fundamental que permite armazenar e manipular uma coleção ordenada de elementos do mesmo tipo. Os vetores são utilizados para representar conjuntos de valores, como números, caracteres ou lógicos.

Criação de vetores:

Existem várias maneiras de gerar vetores em R. A sintaxe básica para criar um vetor é utilizando a função `c()`, que combina elementos em um vetor.

```
# Vetor numérico
vetor_numerico <- c(1, 2, 3, 4, 5)

# Vetor de caracteres
vetor_caracteres <- c("maçã", "laranja", "banana")

# Vetor lógico
vetor_logico <- c(TRUE, FALSE, TRUE)
```

Código-fonte 20 - Criação de vetores
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Você pode criar vetores vazios e adicionar elementos posteriormente. Para criar um vetor vazio, você pode usar a função `vector()`.

```
# Vetor vazio
vetor_vazio <- vector("numeric", length = 0)
```

Código-fonte 21 - Criação de um vetor vazio
Fonte: Elaborado pelo autor (2023)

Acesso aos elementos do vetor:

Em R, você pode acessar elementos específicos de um vetor usando índices. Os índices em R começam em 1. Veja alguns exemplos:

```
# Acesso a elementos individuais
vetor <- c(10, 20, 30, 40, 50)
primeiro_elemento <- vetor[1]   # Retorna o primeiro elemento
(10)
terceiro_elemento <- vetor[3]   # Retorna o terceiro elemento
(30)

# Acesso a um intervalo de elementos
sub_vetor <- vetor[2:4]   # Retorna um sub-vetor contendo os
elementos 20, 30 e 40
```

Código-fonte 22 - Acessando elementos específicos de um vetor usando índices
Fonte: Elaborado pelo autor (2023)

Operações em vetores:

O R oferece uma variedade de operações que podem ser aplicadas a vetores. Alguns exemplos incluem:

```
# Operações numéricas em vetores
vetor1 <- c(1, 2, 3)
vetor2 <- c(4, 5, 6)

soma <- vetor1 + vetor2  # Soma elemento por elemento
produto <- vetor1 * vetor2   # Multiplicação elemento por
elemento

# Operações lógicas em vetores
vetor_logico1 <- c(TRUE, FALSE, TRUE)
vetor_logico2 <- c(FALSE, TRUE, FALSE)

resultado <- vetor_logico1 & vetor_logico2  # Operação lógica
AND
```

Código-fonte 23 - Operações aplicadas a vetores
Fonte: Elaborado pelo autor (2023)

Funções aplicadas a vetores:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Em R, você pode aplicar funções a um vetor para obter informações resumidas sobre seus elementos. Alguns exemplos de funções úteis incluem:

```
vetor <- c(10, 20, 30, 40, 50)

# Comprimento do vetor
comprimento <- length(vetor)

# Valor mínimo e máximo
minimo <- min(vetor)
maximo <- max(vetor)

# Média e soma
media <- mean(vetor)
soma <- sum(vetor)
```

Código-fonte 24 - Funções aplicadas a vetores
Fonte: Elaborado pelo autor (2023)

Manipulação de vetores:

A linguagem R oferece várias funções para adicionar, remover e modificar elementos em um vetor. Algumas dessas funções incluem:

```
# Adicionar elementos a um vetor
vetor <- c(1, 2, 3)
vetor <- c(vetor, 4)  # Adiciona o elemento 4 ao vetor

# Remover elementos de um vetor
vetor <- c(1, 2, 3, 4, 5)
vetor <- vetor[-3]  # Remove o terceiro elemento do vetor

# Modificar elementos em um vetor
vetor <- c(1, 2, 3)
vetor[2] <- 10  # Modifica o segundo elemento para 10
```

Código-fonte 25 - Manipulação de vetores
Fonte: Elaborado pelo autor (2023)

1.8 Matrizes em R

Em R, as matrizes são estruturas de dados bidimensionais que permitem armazenar uma coleção de elementos organizados em linhas e colunas. As matrizes são úteis para representar dados tabulares, como tabelas ou conjuntos de valores numéricos.

Criação de matrizes:

Em R, você pode criar uma matriz utilizando a função `matrix()`. A sintaxe básica para criar uma matriz é:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
nome_matriz <- matrix(data, nrow, ncol)
```

Código-fonte 26 - Criação de matrizes
Fonte: Elaborado pelo autor (2023)

Na qual:

- `data` é um vetor de elementos que serão organizados na matriz.
- `nrow` é o número de linhas da matriz.
- `ncol` é o número de colunas da matriz.

Aqui está um exemplo de criação de uma matriz numérica 3x3 em R:

```
matriz_numerica <- matrix(c(1, 2, 3, 4, 5, 6, 7, 8, 9), nrow =
3, ncol = 3)
```

Código-fonte 27 - Exemplo de criação de uma matriz numérica 3x3 em R
Fonte: Elaborado pelo autor (2023)

Você também pode criar matrizes vazias e preenchê-las posteriormente. Para criar uma matriz vazia, basta utilizar a função `matrix()` com os argumentos `nrow` e `ncol` definidos, mas sem especificar o argumento `data`. Por exemplo:

```
matriz_vazia <- matrix(nrow = 2, ncol = 2)
```

Código-fonte 28 - Criação de matrizes vazias preenchidas posteriormente
Fonte: Elaborado pelo autor (2023)

Acesso aos elementos da matriz:

Você pode acessar elementos específicos de uma matriz em R usando índices para referenciar as linhas e colunas. Os índices em R começam em 1. Veja alguns exemplos:

```
matriz <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, ncol = 3)

# Acesso a elementos individuais
elemento <- matriz[1, 2]  # Retorna o elemento na primeira linha
e segunda coluna

# Acesso a uma linha específica
linha <- matriz[2, ]  # Retorna a segunda linha inteira como um
vetor

# Acesso a uma coluna específica
coluna <- matriz[, 3]  # Retorna a terceira coluna inteira como
um vetor
```

Código-fonte 29 - Elementos da Matriz
Fonte: Elaborado pelo autor (2023)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Operações em matrizes:

A linguagem R oferece uma variedade de operações que podem ser aplicadas a matrizes. Alguns exemplos incluem:

```
matriz1 <- matrix(c(1, 2, 3, 4)
, nrow = 2, ncol = 2)
matriz2 <- matrix(c(5, 6, 7, 8), nrow = 2, ncol = 2)

# Soma de matrizes
soma <- matriz1 + matriz2

# Multiplicação de matrizes
multiplicacao <- matriz1 %*% matriz2

# Transposição de uma matriz
transposta <- t(matriz1)
```

Código-fonte 30 - Operações em Matrizes
Fonte: Elaborado pelo autor (2023)

Funções aplicadas a matrizes:

Em R, você pode aplicar funções a matrizes para obter informações resumidas sobre seus elementos. Alguns exemplos de funções úteis incluem:

```
matriz <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, ncol = 3)

# Comprimento de uma dimensão da matriz
comprimento_linhas <- nrow(matriz)
comprimento_colunas <- ncol(matriz)

# Valor mínimo e máximo da matriz
minimo <- min(matriz)
maximo <- max(matriz)

# Soma de elementos da matriz
soma <- sum(matriz)
```

Código-fonte 31 - Funções aplicadas a matrizes
Fonte: Elaborado pelo autor (2023)

Manipulação de matrizes:

O R oferece várias funções para adicionar, remover e modificar elementos em matrizes. Algumas dessas funções incluem:

```
matriz <- matrix(c(1, 2, 3, 4), nrow = 2, ncol = 2)

# Adicionar uma nova linha à matriz
nova_linha <- c(5, 6)
matriz <- rbind(matriz, nova_linha)
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Adicionar uma nova coluna à matriz
nova_coluna <- c(7, 8)
matriz <- cbind(matriz, nova_coluna)

# Remover uma linha da matriz
matriz <- matriz[-2, ]

# Remover uma coluna da matriz
matriz <- matriz[, -3]
```

Código-fonte 32 - Manipulação de matrizes
Fonte: Elaborado pelo autor (2023)

1.9 Instalação e Manipulação de Pacotes

Os pacotes em R são extensões que adicionam funcionalidades à linguagem, permitindo que os usuários realizem tarefas específicas, como Visualização de Dados, Modelagem Estatística Avançada, Aprendizado de Máquina, entre outras.

Instalando pacotes:

Antes de usar um pacote em R, é necessário instalá-lo. O R possui um repositório central chamado CRAN (Comprehensive R Archive Network), que contém uma vasta coleção de pacotes disponíveis para instalação. Para instalar um pacote, você pode utilizar a função `install.packages()` seguida pelo nome do pacote desejado.

```
install.packages("nome_do_pacote")
```

Código-fonte 33 - Instalando pacotes
Fonte: Elaborado pelo autor (2023)

Após a execução do comando, o R irá baixar e instalar o pacote especificado. Você precisará fazer isso apenas uma vez, a menos que deseje atualizar o pacote posteriormente.

Carregando pacotes:

Assim que um pacote tiver sido instalado, você precisa carregá-lo na sessão do R para poder utilizá-lo. Isso é feito com a função `library()` ou `require()`. Ao executar o comando, o pacote será carregado e todas as funções e recursos fornecidos pelo pacote estarão disponíveis para uso.

```
library(nome_do_pacote)
```

Código-fonte 34 - Carregando pacotes
Fonte: Elaborado pelo autor (2023)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Explorando pacotes:

Após carregar um pacote, é útil explorar as funcionalidades e os recursos que ele oferece. Existem várias maneiras de fazer isso:

- **Documentação do pacote:** a maioria dos pacotes possui documentação detalhada disponível online ou pode ser acessada diretamente no R. Você pode utilizar a função `help()` ou `?` seguido do nome da função específica que deseja explorar.

```
help(nome_do_pacote)  # Mostra a documentação do pacote
?nome_da_funcao   # Mostra a documentação de uma função específica
```

Código-fonte 35 - Explorando pacotes
Fonte: Elaborado pelo autor (2023)

- **Listando funções:** para obter uma lista das funções disponíveis em um pacote, você pode usar a função `ls()` ou `objects()`.

```
ls("package:nome_do_pacote")  # Lista as funções do pacote
objects("package:nome_do_pacote")    # Lista os objetos disponíveis no pacote
```

Código-fonte 36 - Listando funções
Fonte: Elaborado pelo autor (2023)

- **Exemplos e tutoriais:** muitos pacotes incluem exemplos e tutoriais para ajudar os usuários a começar a utilizar suas funcionalidades. Esses exemplos podem ser encontrados na documentação do pacote ou em recursos online, como sites e blogs especializados.

Atualizando pacotes:

Periodicamente, novas versões de pacotes são lançadas para corrigir bugs, adicionar recursos ou melhorar o desempenho. Para atualizar pacotes instalados, você pode usar a função `update.packages()`. Este comando verificará se há atualizações disponíveis para os pacotes instalados e os atualizará para suas versões mais recentes.

```
update.packages()
```

Código-fonte 37 - Atualizando pacotes
Fonte: Elaborado pelo autor (2023)

Removendo pacotes:

Se você não precisar mais de um pacote, pode removê-lo do seu sistema usando a função `remove.packages()`. Este comando desinstalará completamente o pacote do seu sistema.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
remove.packages("nome_do_pacote")
```

Código-fonte 38 - Removendo pacotes
Fonte: Elaborado pelo autor (2023)

1.10 Dataframes e tibbles

No R, tanto os dataframes quanto os tibbles são estruturas de dados amplamente utilizadas para armazenar e manipular conjuntos de dados tabulares. Ambas as estruturas são bidimensionais, organizadas em linhas e colunas e permitem o armazenamento de diferentes tipos de dados em cada coluna.

Dataframes:

Os dataframes correspondem a uma das estruturas de dados mais comuns em R por serem semelhantes a matrizes, mas com a capacidade de armazenar colunas com tipos de dados diferentes. Cada coluna de um dataframe pode ser de um tipo diferente, como numérico, caractere, lógico, fator, entre outros. Para criar um dataframe em R, você pode usar a função `data.frame()`. A sintaxe básica é a seguinte:

```
nome_dataframe <- data.frame(coluna1, coluna2, coluna3, ...)
```

Código-fonte 39 - Criação de um dataframe em R (sintaxe básica)
Fonte: Elaborado pelo autor (2023)

Aqui está um exemplo de criação de um dataframe simples em R:

```
# Criando um dataframe
df <- data.frame(
  nome = c("João", "Maria", "Pedro"),
  idade = c(25, 30, 28),
  casado = c(TRUE, FALSE, TRUE)
)
```

Código-fonte 40 - Criação de um dataframe simples em R
Fonte: Elaborado pelo autor (2023)

No exemplo de criação de um dataframe simples em R, criamos um dataframe chamado `df` com três colunas: "nome", "idade" e "casado". Cada coluna é definida por um vetor correspondente.

Tibbles:

Os tibbles são uma versão moderna e aprimorada dos dataframes em R, introduzidos pelo pacote `tibble`. Os tibbles têm muitas características semelhantes aos dataframes, mas com algumas diferenças sutis. Eles são projetados para serem mais intuitivos e

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

amigáveis, fornecendo uma experiência de programação mais consistente. Para criar um tibble em R, você pode usar a função `tibble()`, disponível no pacote `tibble`. A sintaxe básica é a seguinte:

```
nome_tibble <- tibble(coluna1, coluna2, coluna3, ...)
```

Código-fonte 41 - Tibbles
Fonte: Elaborado pelo autor (2023)

Aqui está um exemplo de criação de um tibble simples em R:

```
# Criando um tibble
library(tibble)

tib <- tibble(
  nome = c("João", "Maria", "Pedro"),
  idade = c(25, 30, 28),
  casado = c(TRUE, FALSE, TRUE)
)
```

Código-fonte 42 - Criação de um tibble simples em R
Fonte: Elaborado pelo autor (2023)

No exemplo, criamos um tibble chamado `tib` com três colunas: "nome", "idade" e "casado". Assim como nos dataframes, cada coluna é definida por um vetor correspondente.

Acesso aos elementos de dataframes e tibbles:

Você pode acessar elementos específicos de dataframes e tibbles em R usando índices para referenciar linhas e colunas. Os índices em R começam em 1. Veja alguns exemplos:

```
# Acesso a elementos individuais
elemento_df <- df[1, 2]  # Retorna o elemento da primeira linha
e segunda coluna do dataframe
elemento_tib <- tib[2, 3]  # Retorna o elemento da segunda linha
e terceira coluna do tibble

# Acesso a uma coluna específica
coluna_df <- df$idade  # Retorna a coluna "idade" do dataframe
coluna_tib <- tib$casado  # Retorna a coluna "casado" do tibble
```

Código-fonte 43 - Acesso aos elementos de dataframes e tibbles
Fonte: Elaborado pelo autor (2023)

Manipulação de dataframes e tibbles:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Entendendo a ciência de dados

Dataframes e tibbles em R oferecem uma variedade de funções e operações para manipulação de dados, como adicionar ou remover colunas, filtrar linhas com base em critérios, realizar agregações, entre outros. Algumas das funções mais comuns incluem:

```
     # Adicionar uma nova coluna
     df$nova_coluna <- c(1, 2, 3)  # Adiciona a coluna "nova_coluna"
ao dataframe
     tib$nova_coluna <- c(1, 2, 3)  # Adiciona a coluna "nova_coluna"
ao tibble

     # Remover uma coluna
     df <- subset(df, select = -nome)  # Remove a coluna "nome" do
dataframe
     tib <- tib[, -2]  # Remove a segunda coluna do tibble

     # Filtrar linhas com base em critérios
     df_filtrado <- subset(df, idade > 25)  # Filtra as linhas do
dataframe onde a idade é maior que 25
     tib_filtrado <- filter(tib, idade > 25)  # Filtra as linhas do
tibble onde a idade é maior que 25
```

Código-fonte 44 - Manipulação de dataframes e tibbles
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

IHAKA, R., GENTLEMAN, R. R: A language for data analysis and graphics. **Journal of Computational and Graphical Statistics**, 5(3), p. 299-314, 1996.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com